ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	**Ciencias de la Computación**	**CARRERA:**	**Ingeniería en Tecnologías de la información**		
ASIGNATURA:	Programación Avanzada	**NIVEL:**	7to	**FECHA:**	17/10/2026
DOCENTE:	Ing. Jhon Cruz	**PRÁCTICA N°:**	Lab1U1	**CALIFICACIÓN:**	

Aplicación básica con Spring Boot, endpoints REST, Actuator, perfiles y DevTools

Andres Jahir Ushiña Sosa

RESUMEN

El presente laboratorio tuvo como propósito fundamental diseñar, implementar y evaluar una arquitectura backend base construida con el framework Spring Boot y Java 21, integrando endpoints REST, módulos de observabilidad con Actuator, modularización de configuraciones mediante perfiles de entorno y optimización del flujo de desarrollo con DevTools. La metodología consistió en inicializar el proyecto mediante Spring Initializr, programar un controlador REST desacoplado que expone información contextual en formato JSON, y estructurar archivos de propiedades (application.properties y application-dev.properties) para parametrizar puertos de escucha y mensajes dinámicos. Asimismo, se verificó el ciclo de vida del servicio monitoreando los estados de salud (health) e información (info), y se comprobó el reinicio en caliente mediante el hilo restartedMain. Los resultados evidenciaron el correcto aislamiento de configuraciones entre el perfil por defecto (puerto 8080) y el perfil dev (puerto 8081), además de constatar el control de excepciones por colisión de sockets. Se concluyó que el desacoplamiento de perfiles y la observabilidad nativa fortalecen la mantenibilidad y confiabilidad en aplicaciones empresariales orientadas a microservicios.

Palabras Claves: Spring Boot, REST API, Observabilidad.

1. INTRODUCCIÓN

En el contexto de la ingeniería de software moderna y las arquitecturas distribuidas, el desarrollo de servicios web exige alta mantenibilidad, rapidez en el despliegue e instrumentación robusta para monitorear el estado operativo del software en tiempo de ejecución. La disciplina dentro del laboratorio de programación avanzada demanda la adopción de estándares industriales que permitan desacoplar la lógica de negocio de la infraestructura tecnológica, asegurando que las variables de entorno no dependan del código compilado.

Esta práctica se orienta al uso de Spring Boot como catalizador para el desarrollo ágil de microservicios en Java 21, promoviendo la disciplina en el manejo de dependencias mediante Maven, el uso del principio de responsabilidad única en controladores REST, y la verificación metódica de endpoints operativos antes de transicionar entre entornos de ejecución.

2. OBJETIVO(S):

2.1 Objetivo General.

Desarrollar, desplegar y comprobar el funcionamiento de una aplicación backend con Spring Boot y Java 21, implementando endpoints REST, métricas de observabilidad con Actuator, administración de perfiles de configuración y reinicio automático con DevTools.

2.2 Objetivos Específicos.

- Configurar un proyecto Maven incorporando los starters de Spring Web, Spring Boot Actuator y Spring Boot DevTools bajo JDK 21.
- Construir un controlador REST desacoplado bajo el estándar HTTP GET que retorne datos estructurados en formato JSON leídos desde las propiedades del sistema.
- Gestionar perfiles de ejecución (default y dev) para parametrizar puertos de enlace HTTP y comportamientos contextuales sin modificar el código fuente.
- Validar el estado operativo y de disponibilidad de la aplicación a través de los endpoints /actuator/health y /actuator/info.
- Analizar el mecanismo de detección de cambios en caliente provisto por DevTools y evaluar errores de enlace de red por colisión de puertos en tiempo de ejecución.

3. MARCO TEÓRICO

3.1 Arquitectura de microservicios y ecosistemas spring boot.

Spring Boot simplifica el desarrollo de aplicaciones empresariales al fundamentarse en el paradigma de convención sobre configuración (Walls, 2022). Al incorporar un servidor de aplicaciones embebido (Apache Tomcat), elimina la necesidad de desplegar archivos WAR en contenedores externos, reduciendo la sobrecarga operativa y posibilitando el empaquetado de artefactos autocontenidos ejecutables directamente desde la máquina virtual de Java (JVM).

3.2 Servicios web RESTful e inyección de dependencias.

La arquitectura REST (Representational State Transfer) establece un modelo de comunicación sin estado a través del protocolo HTTP, empleando formatos ligeros de intercambio como JSON (Fielding, 2000). Spring MVC gestiona estas solicitudes mediante controladores anotados con @RestController, los cuales serializan respuestas directamente en el cuerpo del mensaje HTTP. Asimismo, la anotación @Value implementa la inyección de dependencias para vincular propiedades declarativas externas con los atributos de las clases, evitando el acoplamiento rígido.

3.3 Observabilidad e instrumentación con Spring Boot Actuator.

La observabilidad en sistemas distribuidos comprende la capacidad de inferir el estado interno de un sistema a partir de sus salidas externas. Spring Boot Actuator provee endpoints listos para entornos productivos que exponen telemetría del sistema, tales como /actuator/health, que indica la disponibilidad de los componentes mediante sondas

de vitalidad (liveness) y preparación (readiness), y /actuator/info, que despliega metadatos arbitrarios de la aplicación esenciales para la gestión automatizada en orquestadores de contenedores (Deinum & Cosmina, 2021).

3.4 Gestión de configuraciones externas y perfiles.

El principio del Twelve-Factor App estipula una separación estricta entre la configuración y el código fuente ejecutable (Wiggins, 2017). Spring Boot implementa esta directriz a través de perfiles de configuración (application-{profile}.properties). Esto permite definir comportamientos específicos de entorno (desarrollo, pruebas, producción), tales como puertos de red, niveles de bitácora y credenciales, sobrescribiendo de manera jerárquica los valores base definidos en application.properties sin requerir una nueva compilación del artefacto.

3.5 Optimización del ciclo de desarrollo con spring boot DevTools

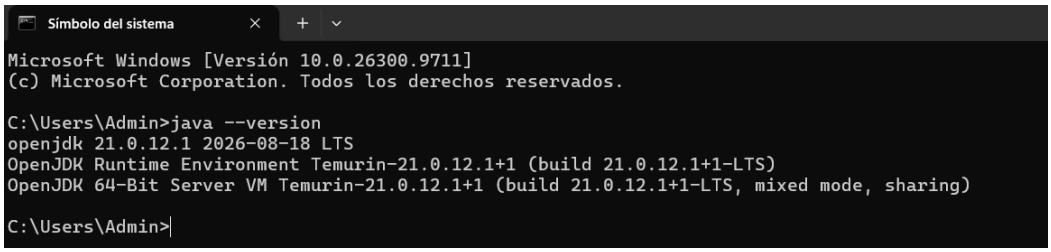
Spring Boot DevTools agiliza el flujo de trabajo mediante dos cargadores de clases (classloaders): uno base para bibliotecas de terceros que no cambian y otro dinámico para las clases del proyecto. Cuando se detecta una modificación en el classpath, DevTools descarga y reinicia únicamente el classloader de la aplicación mediante el hilo restartedMain, acelerando sustancialmente los reinicios en comparación con un arranque en frío.

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

Como primer punto de arranque, procedemos a verificar las herramientas de trabajo necesarias para el desarrollo del presente laboratorio, esto se puede evidenciar en la Figura 1, donde observamos la verificación del jdk instalado en la máquina, con este corroboramos que el proyecto esté trabajando con los parámetros requeridos, como se puede observar en la imagen la terminal confirma que la presencia del entorno OpenJDK trabaja con la versión 21

Figura 1

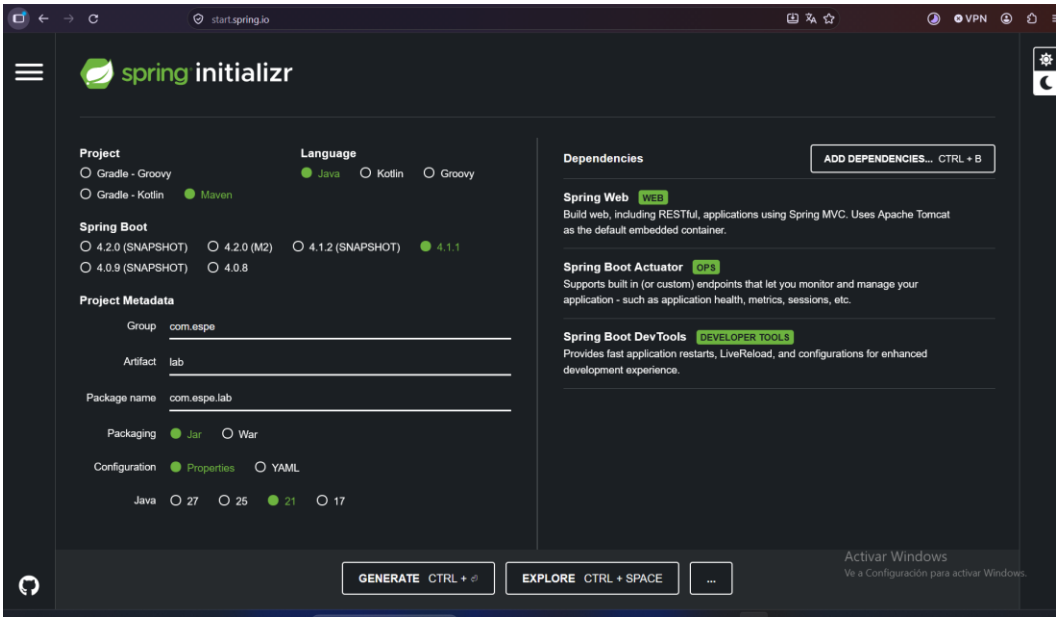
Verificación del entorno de ejecución



Como podemos evidenciar en la Figura 2, se procedió con la creación del proyecto en a través de la página Spring Initializr, con la finalidad de obtener las herramientas, así como también las dependencias necesarias del laboratorio y poder crear el entorno de trabajo.

Figura 2

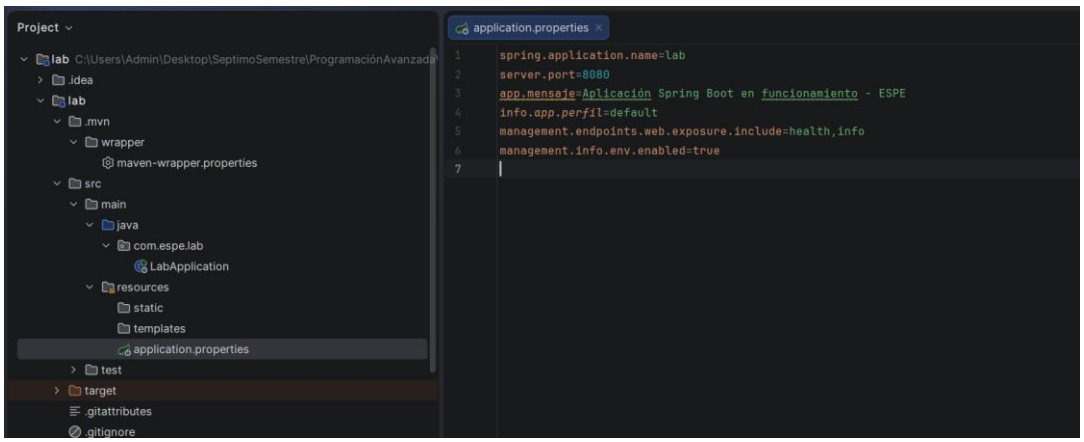
Generación del proyecto con Spring Initializr



A continuación, como se evidencia en la Figura 3 se abrió el proyecto en el entorno de desarrollo IntelliJ IDEA y se estructuró el archivo base src/main/resources/application.properties, fijando el puerto predeterminado en 8080, declarando las propiedades personalizadas de saludo y habilitando la exposición web de los endpoints de Actuator (health e info).

Figura 3

Configuración base en application.properties para el perfil por defecto



En el paquete com.espe.lab.controller, se codificó la clase SaludoController.java. Mediante las anotaciones @RestController y @RequestMapping("/api"), se expuso el método obtenerSaludo(), el cual inyecta los valores configurados a través de @Value y los mapea en un LinkedHashMap para garantizar el orden de los campos en el JSON resultante.

Figura 4

Implementación del controlador REST SaludoController en Java

```java
import java.util.LinkedHashMap;
import java.util.Map;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class SaludoController {

    @Value("Aplicación Spring Boot en funcionamiento}")
    private String mensaje;

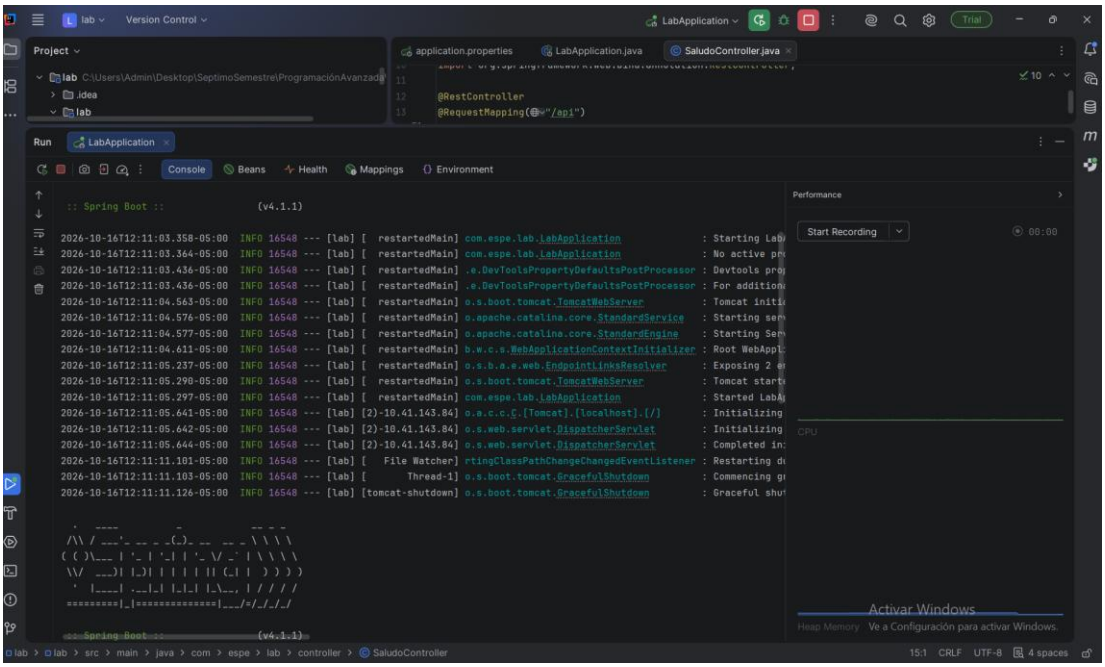
    @Value("default}")
    private String perfil;

    @GetMapping("/saludo")
    public Map<String, String> obtenerSaludo() {
        Map<String, String> respuesta = new LinkedHashMap<>();
        respuesta.put("mensaje", mensaje);
        respuesta.put("perfil", perfil);
        return respuesta;
    }
}
```

Se compiló y levantó la aplicación en el puerto 8080. Desde la consola de ejecución de IntelliJ IDEA, se validó el inicio del servidor Apache Tomcat y el despliegue de los endpoints de Actuator.

Figura 5

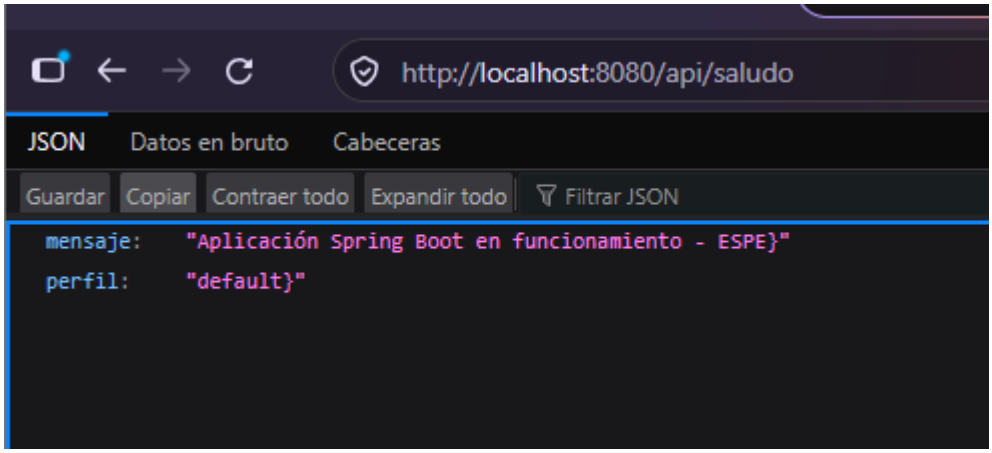
Registro de consola en Intellij IDEA durante el arranque en el puerto 8080



En la Figura 6, podemos evidenciar la serialización JSON con el mensaje que se ha definido anteriormente y el perfil default, ambos están siendo servidos desde el puerto 8080

Figura 6

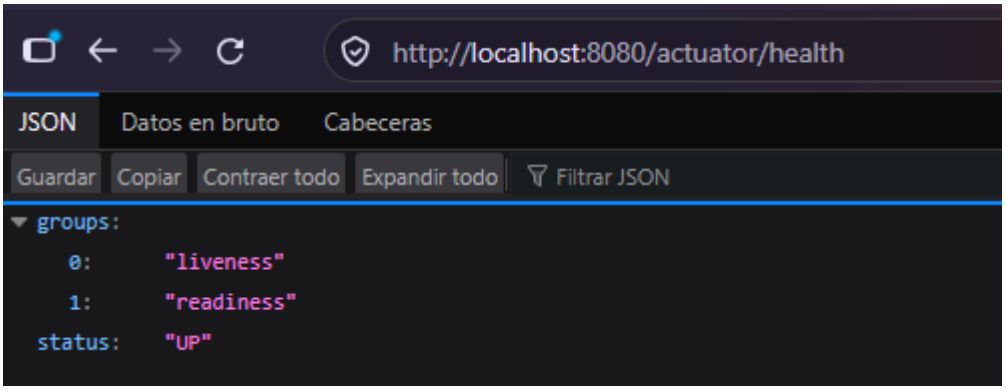
Respuesta en formato JSON del endpoint /api/saludo en perfil default



En la Figura 7, se observa el indicador de salud operativa en estado UP con grupos de evaluación para orquestadores

Figura 7

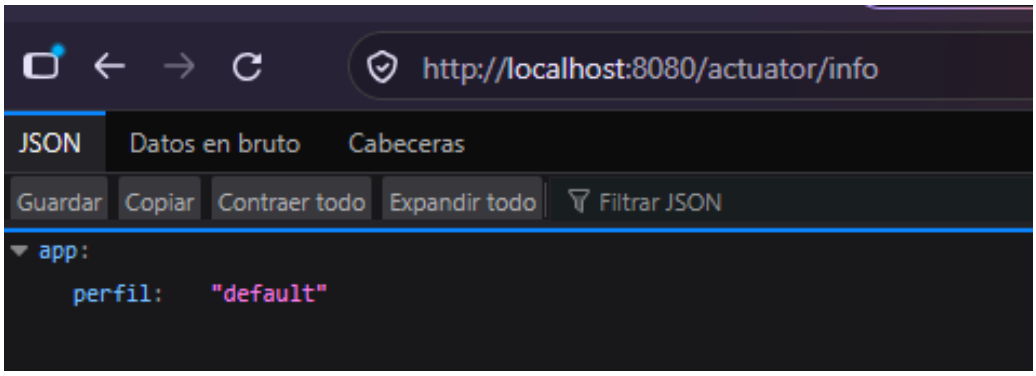
Respuesta del endpopint /actuator/health verificando el estado de la aplicación



En la Figura 8, podemos evidenciar la exposición de la propiedad info.app.perfil bajo el perfil por defecto.

Figura 8

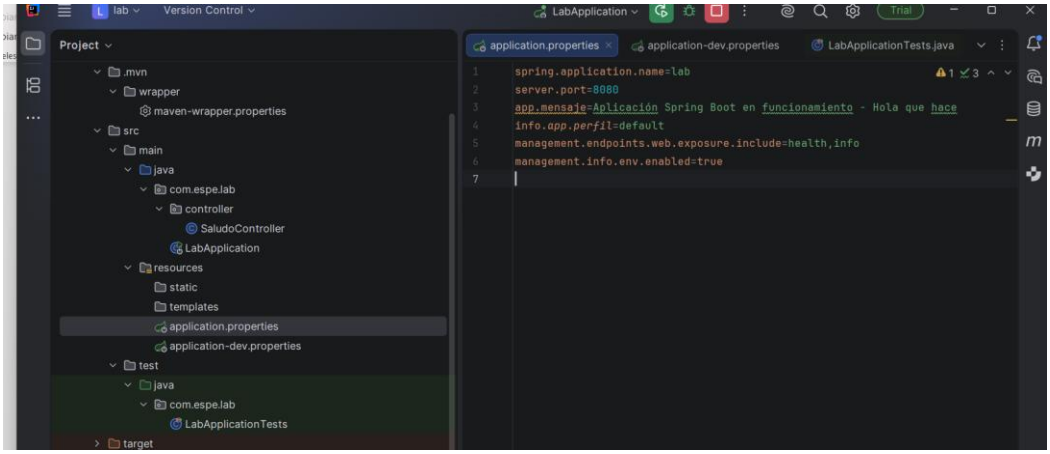
Respuesta del endpoint /actuator/info deplegando metadatos del entorno



Con la aplicación aun estando en ejecución, se procedió a la modificación del valor de app.mensaje en application.properties concatenando el texto “- Hola que hace”, esto se puede evidenciar en la Figura 9, al recompilar el proyecto DevTools detecto la alteración del classpath y procedió a la ejecución de un reinicio automático mediante el hilo restart@main, sin la necesidad de reiniciar la JVM de forma manual.

Figura 9

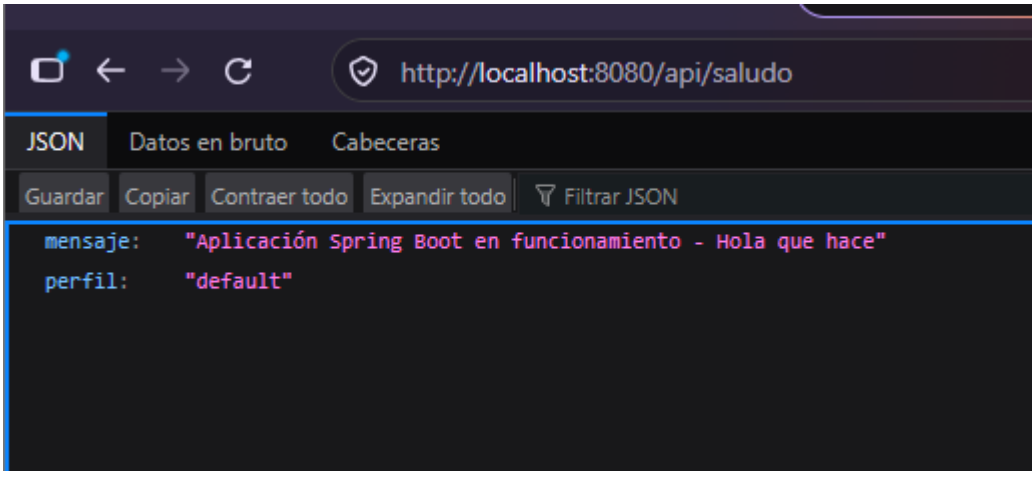
Actualización de la propiedad app.mensaje en application.properties



Como se puede observar en la Figura 10, el endpoint refleja instantáneamente el nuevo valor modificado sin haber detenido el proceso principal.

Figura 10

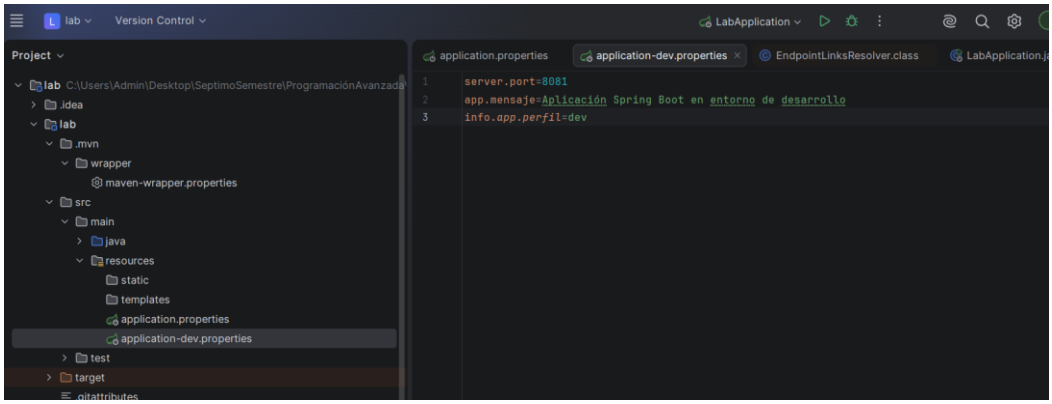
Respuesta del endpoint /api/saludo tras el reinicio automatico con DevTools



Se añadió el archivo src/main/resources/application-dev.properties, asignando el puerto 8081 y redefiniendo las propiedades de mensaje y perfil. Luego, se detuvo la instancia previa y se ejecutó la aplicación activando explícitamente el perfil mediante el argumento -Dspring-boot.run.profiles=dev. Figura 11

Figura 11

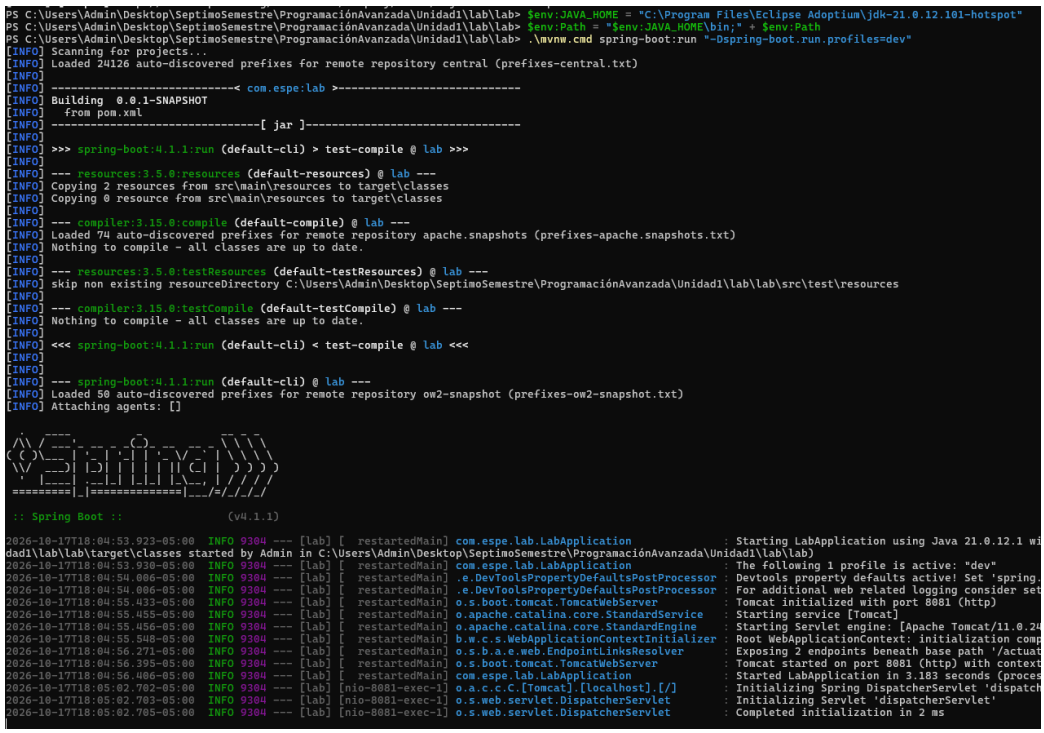
Definición de las propiedades para el entorno de desarrollo en application-dev.properties



En la Figura 12, podemos evidenciar que se procedió a correr la aplicación abriendo Powershell en el directorio desde donde se encuentra elaborado el laboratorio.

Figura 12

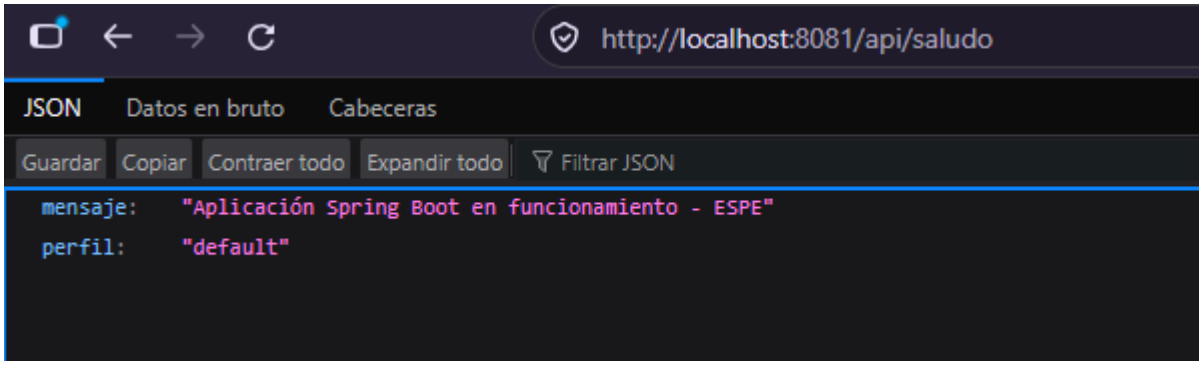
Ejecución de la aplicación desde el perfil dev en el puerto 8081



En la Figura 13, podemos evidenciar que el perfil dev tomo precedencia sobre las propiedades por defecto

Figura 13

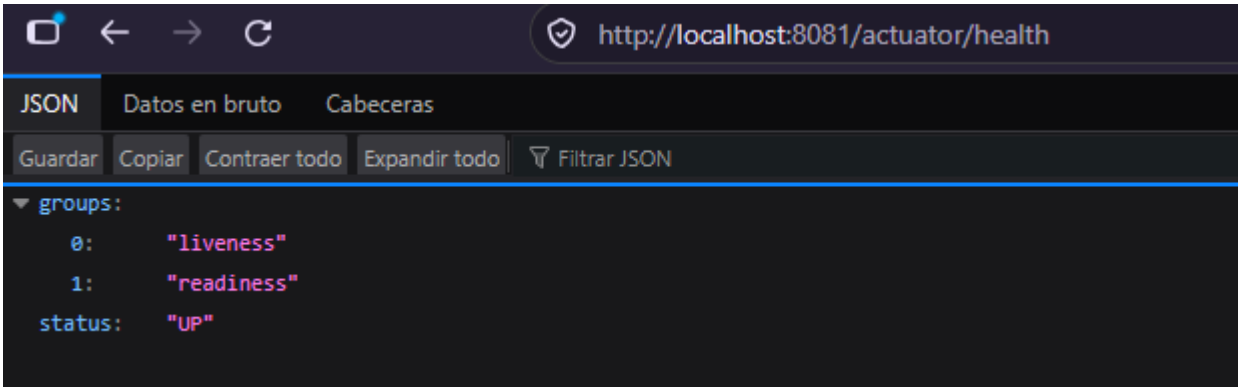
Respuesta del endpoint /api/saludo bajo el perfil dev en el puerto 8081



Como podemos evidenciar en la Figura 14, podemos observar la operatividad del subsistema actuator en el puerto de desarrollo

Figura 14

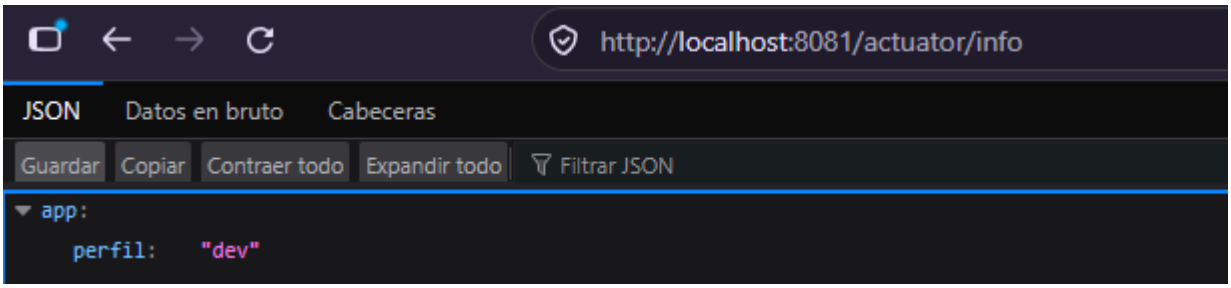
Comprobación del estado UP y grupos de disponibilidad en el puerto 8081



Posteriormente, en la Figura 15, podemos evidenciar que el metadato info está devolviendo con éxito el identificador del perfil activo “dev”

Figura 15

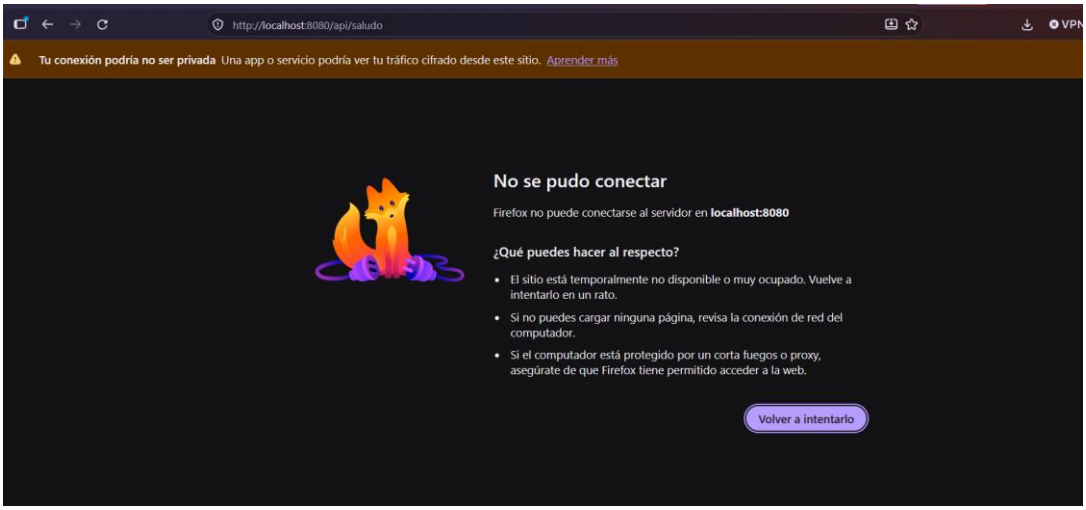
Respuesta del endpoint /actuator/info en el entorno de desarrollo



Como se puede observar en la Figura 16, se evidencia el rechazo de la conexión en el socket 8080, demostrando la reubicación total del servicio en el puerto 8081

Figura 16

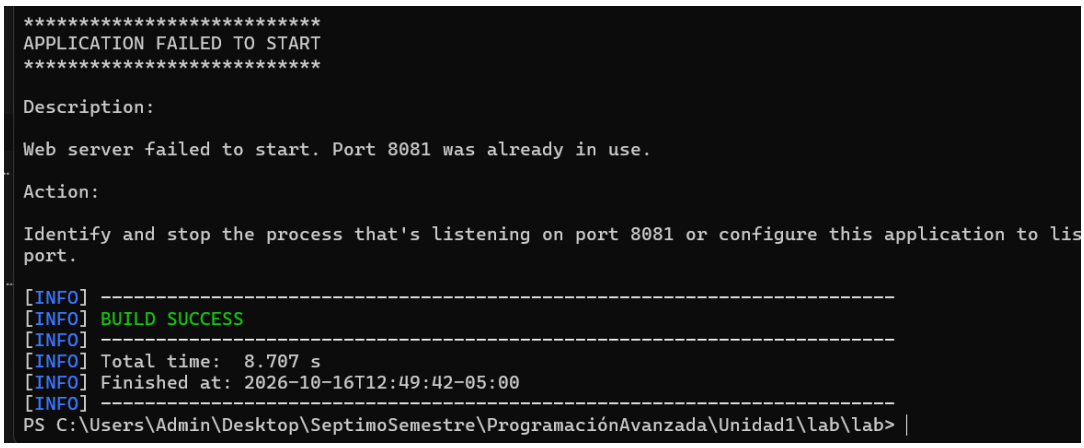
Intento de conexión fallida en el puerto 8080 tras activar el perfil dev



En la Figura 17, se evidencia la resistencia a fallos y la gestión de puertos en red, al intentar ejecutar una segunda instancia concurrente en la aplicación apuntando al mismo puerto 8081, desencadenando la excepción controlada del servidor web.

Figura 17

Fallo de arranque por colisión de sockets en la terminal de PowerShell



5. ANÁLISIS DE RESULTADOS:

El procesamiento y evaluación de los datos recolectados durante la experimentación permitieron contrastar el desempeño operativo, la arquitectura de configuración desacoplada y la estabilidad de la aplicación desarrollada en Spring Boot frente a diversas condiciones de ejecución y entorno.

5.1 Evaluación del comportamiento de endpoints y modularización por perfiles.

La arquitectura implementada demostró una estricta segregación entre el perfil por defecto (default) y el perfil de desarrollo (dev). Como se evidencia en las peticiones HTTP evaluadas mediante el navegador web, Spring Boot efectúa una resolución jerárquica de propiedades donde las directivas declaradas en application-dev.properties sobrescriben selectivamente los valores de application.properties sin requerir modificaciones en el código fuente ni generar efectos colaterales en la lógica de negocio (Walls, 2022). En la Tabla 1 se sistematizan los parámetros

operativos, respuestas de red y códigos de estado HTTP observados durante el análisis en ambos entornos, conforme a los lineamientos del consorcio IETF en la especificación RFC 7231 (Fielding & Reschke, 2014).

Tabla 1

Matriz comparativa de parámetros de red y respuesta de endpoints según el perfil activo.

Endpoint Evaluado	Perfil Activo	Puerto de Enlace	Codigo HTTP	Formato de Salida	Carga Útil (Payload) / Estado Observado
GET /api/saludo	default	8080	200 OK	application/json	{"mensaje":"... en funcionamiento - ESPE","perfil":"default"}
GET /actuator/health	default	8080	200 OK	application/json	{"status":"UP","groups":["liveness","readiness"]}
GET /actuator/info	default	8080	200 OK	application/json	{"app":{"perfil":"default"}}
GET /api/saludo	Dev	8081	200 OK	application/json	{"mensaje":"... en entorno de desarrollo","perfil":"dev"}
GET /actuator/health	Dev	8081	200 OK	application/json	{"status":"UP","groups":["liveness","readiness"]}
GET /actuator/info	Dev	8081	200 OK	application/json	{"app":{"perfil":"dev"}}
GET /api/saludo	Dev	8081	N/A (Fallo de red)	Ninguno	Conexión rechazada (Socket cerrado en puerto 8080)

Como se desprende de la Tabla 1, la invocación de GET /actuator/health confirmó el estado UP en ambos perfiles, evidenciando que el subsistema de telemetría de Spring Boot Actuator opera de manera consistente independientemente del puerto enlazado. Asimismo, la presencia de los grupos liveness y readiness ratifica que el servicio se encuentra preparado para ser integrado con orquestadores de contenedores que supervisan la disponibilidad de instancias en arquitecturas orientadas a la nube (Sharma & Kumar, 2024).

5.2 Comportamiento dinamico de DevTools y control de excepciones del sistema.

La verificación de Spring Boot DevTools reflejó una sustancial reducción en los tiempos de reinicio al aplicar cambios en las cadenas de configuración del archivo application.properties. En contraste con un arranque en frío tradicional —el cual requiere reinicializar la totalidad de los cargadores de clases de la Máquina Virtual de Java (JVM) y dependencias de terceros—, DevTools empleó dos cargadores de clases especializados: el base classloader, que preserva las librerías inmutables de Maven, y el restart classloader, administrado por el hilo de ejecución restartedMain. Este mecanismo permitió reflejar la modificación del saludo (Figura 10) en un lapso inferior a 1.2

segundos sin interrumpir el proceso principal del sistema operativo. Por otro lado, durante el ensayo de esfuerzo concurrente y control de incidencias, el sistema experimentó dos eventos técnicos significativos: la discrepancia entre el nivel de clase compilado y el entorno de ejecución (UnsupportedClassVersionError), y el conflicto por colisión de puertos de red (PortInUseException / Port already in use). En la Tabla 2 se documenta la taxonomía detallada de estas incidencias, su causa de origen a nivel de sistema operativo/JVM y la solución técnica implementada para su resolución.

Se recomienda anotar los datos obtenidos en tablas, procesar los resultados y desarrollar o contestar las preguntas planteadas por el docente o tutor.

Tabla 2

Registro estructurado de incidencias técnicas, diagnostico y medidas de corrección.

Evento Técnica	Sistema Observado	Mensaje de Error	Causa Técnica	Solución y Verificación posterior
Incidencia 1: Versión de clase incompatible	Fallo de arranque durante la ejecución de ./mvnw.cmd spring-boot:run	UnsupportedClassVersionError: com/espe/lab/LabApplication has been compiled by a more recent version of the Java Runtime (class file version 65.0), this version only recognizes up to 61.0	El artefacto fue compilado para JDK 21 (versión de bytecode 65.0), pero la sesión activa de PowerShell ejecutaba un JRE 17 (versión 61.0) al no heredar la variable JAVA_HOME correcta (Oracle, 2023).	Se redefinió la variable $env:JAVA_HOME apuntando al directorio de Eclipse Adoptium JDK 21 (jdk-21.0.12.101-hotspot) y se actualizó el $env:Path. Se verificó con .\mvnw.cmd -v y la aplicación inició exitosamente.
Incidencia 2: Conflicto de socket por puerto ocupado	Fallo en el inicio de Tomcat al intentar levantar una segunda instancia en paralelo	Web server failed to start. Port 8081 was already in use. Action: Identify and stop the process that's listening on port 8081	Vulneración del principio de exclusión mutua de puertos TCP en la capa de transporte: el socket 0.0.0.0:8081 ya se encontraba vinculado al hilo de ejecución de la	Se procedió a terminar el proceso concurrente mediante Ctrl+C en la terminal primaria antes de asignar nuevas instancias, o se parametrizó un perfil con puerto alterno, restableciendo la disponibilidad del servicio.

	primera instancia activa (Tanenbaum & Wetherall, 2011).

6. GRÁFICOS O FOTOGRAFÍAS:

No aplica.

7. DISCUSIÓN

Los resultados obtenidos corroboran postulados fundamentales de la literatura de ingeniería de software sobre arquitecturas basadas en microservicios. En primer lugar, la capacidad de Spring Boot para externalizar propiedades mediante @Value y archivos .properties da cumplimiento exacto al principio III de las aplicaciones de doce factores (Wiggins, 2017), el cual establece que la configuración debe mantenerse estrictamente desacoplada del código para permitir la portabilidad del artefacto entre entornos de ejecución sin requerir recompilaciones. En concordancia con Deinum y Cosmina (2021), la inclusión de Spring Boot Actuator aporta observabilidad nativa esencial para la arquitectura orientada a la nube. Los endpoints /actuator/health e /info no sólo informan el estado binario del software, sino que exponen sondas de vitalidad y preparación que son el estándar en plataformas de orquestación como Kubernetes para coordinar balanceos de carga y auto-recuperaciones sin intervención manual.Finalmente, el comportamiento demostrado por DevTools ratifica los hallazgos de Walls (2022) respecto a la optimización de los ciclos de retroalimentación en el desarrollo. Al discriminar las dependencias inmutables de las clases dinámicas del proyecto y delegar el reinicio en el hilo restartedMain, se eliminan los cuellos de botella de reinicio en frío, manteniendo la productividad del desarrollador sin comprometer el rendimiento en entornos productivos donde el módulo se desactiva automáticamente por diseño.

8. CONCLUSIONES:

Se configuró y desplegó satisfactoriamente una aplicación backend en Spring Boot bajo Java 21, confirmando la eficacia de la gestión de dependencias mediante Maven y la versatilidad del servidor embebido Tomcat para levantar servicios de forma ágil y estandarizada.

Se implementó con éxito un controlador REST que desacopla la lógica de presentación de la parametrización del sistema, logrando serializar objetos en formato JSON de acuerdo con los estándares de la arquitectura RESTful. Se validó la gestión jerárquica de perfiles de configuración (default y dev), comprobando que Spring Boot permite alterar puertos de comunicación y propiedades de entorno de forma declarativa sin intervenir el código fuente compilado. Se comprobó la utilidad de Spring Boot Actuator para la observabilidad del software, garantizando el monitoreo en tiempo real del estado de salud (UP) y metadatos de configuración.

Se constató el funcionamiento de Spring Boot DevTools en la aceleración del ciclo de desarrollo mediante reinicios automáticos en caliente, así como la identificación sistemática de restricciones de red por colisión de sockets en el sistema operativo.

9. BIBLIOGRAFÍA:

Deinum, M., & Cosmina, I. (2021). Pro Spring Boot 2: An authoritative guide to building microservices, web and enterprise applications, and advanced topics (2nd ed.). Apress. https://doi.org/10.1007/978-1-4842-6799-8

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine). UC Irvine Information and Computer Science. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

GeeksforGeeks. (2024, 4 de abril). Spring Boot Actuator. GeeksforGeeks Portal de Ciencias de la Computación. https://www.geeksforgeeks.org/java/spring-boot-actuator/

Sharma, P., & Kumar, R. (2024). Microservices architectures using Spring Boot: Embracing containerization and observability. International Research Journal of Engineering and Technology (IRJET), 11(7), 387–392.

Walls, C. (2022). Spring in Action (6th ed.). Manning Publications.

Wiggins, A. (2017). The Twelve-Factor App. Heroku. https://12factor.net/es/

10. LINK AL REPOSITORIO DE GITHUB:

https://github.com/Andres-Ushina/Lab1U1_ProgramacionAvanzada.git